

```

91.         sys.exit()
92.         if response in (w + a + s + d).replace(' ', ''):
93.             return response
94.
95.
96. def makeMove(board, move):
97.     """Carry out the given move on the given board."""
98.     # Note: This function assumes that the move is valid.
99.     bx, by = findBlankSpace(board)
100.
101.     if move == 'W':
102.         board[bx][by], board[bx][by+1] = board[bx][by+1], board[bx][by]
103.     elif move == 'A':
104.         board[bx][by], board[bx+1][by] = board[bx+1][by], board[bx][by]
105.     elif move == 'S':
106.         board[bx][by], board[bx][by-1] = board[bx][by-1], board[bx][by]
107.     elif move == 'D':
108.         board[bx][by], board[bx-1][by] = board[bx-1][by], board[bx][by]
109.
110.
111. def makeRandomMove(board):
112.     """Perform a slide in a random direction."""
113.     blankx, blanky = findBlankSpace(board)
114.     validMoves = []
115.     if blanky != 3:
116.         validMoves.append('W')
117.     if blankx != 3:
118.         validMoves.append('A')
119.     if blanky != 0:
120.         validMoves.append('S')
121.     if blankx != 0:
122.         validMoves.append('D')
123.
124.     makeMove(board, random.choice(validMoves))
125.
126.
127. def getNewPuzzle(moves=200):
128.     """Get a new puzzle by making random slides from a solved state."""
129.     board = getNewBoard()
130.
131.     for i in range(moves):
132.         makeRandomMove(board)
133.     return board
134.
135.
136. # If this program was run (instead of imported), run the game:
137. if __name__ == '__main__':
138.     main()

```

After entering the source code and running it a few times, try making experimental changes to it. On your own, you can also try to figure out how to do the following:

- Create a more difficult 5×5 variant of the sliding tile puzzle.